CSCI 677: Advanced Computer Vision - Fall 2026

Instructor: Prof. Nevatia

Assignment # 3

Due on October 27, 2026 before 15:00 PDT

This is a programming assignment to create, train and test a CNN and a transformer network for the task of image classification. To keep the task manageable, we will use a small dataset.

CNN Architecture

You are asked to construct and experiment with a relatively small residual CNN known as ResNet-18. The architecture of ResNet-18 is shown in Figure 1; the figure includes the tensor shapes (spatial resolution and # of channels) at each block as well as filter sizes, number of channels and strides for the convolution layers. Downsampling is performed by stride-2 convolutions in the first residual block of stages 2–4, as shown in Figure 1. No additional pooling is applied between stages. Global average pooling is applied after the final stage.

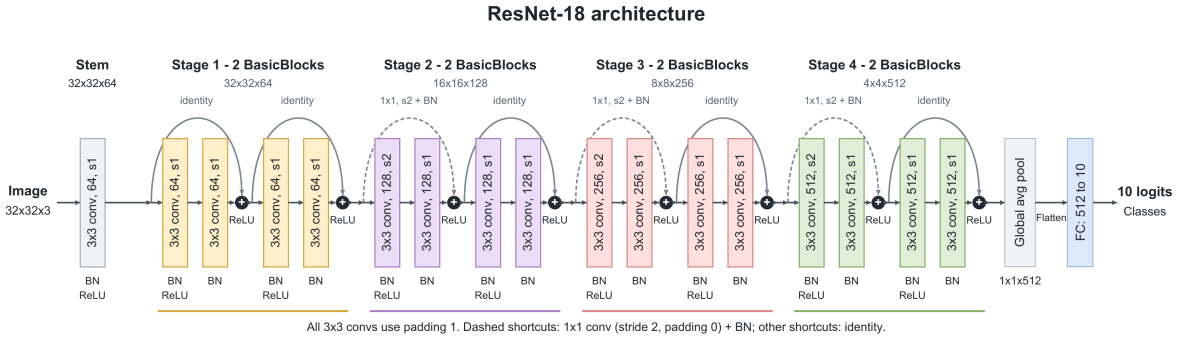


Figure 1: ResNet-18 Architecture

Please define the network architecture like:

```
class ResNet18(nn.Module):
    def __init__(self, num_classes=10):
        super(ResNet18, self).__init__()
        # TODO
    def forward(self, x):
        # TODO
```

ViT Architecture

For this part, you will fine-tune a pre-trained Vision Transformer (ViT) model for our 10-class dataset. Using the Hugging Face transformers library, load the ViTForImageClassification model as demonstrated in the provided PyTorch tutorial slides. To adapt the model to our dataset, you must replace its original classification head by passing the argument num_labels=10 during the from_pretrained() call; this will automatically discard the old head and add a new, randomly initialized classification layer with 10 outputs, making it ready for fine-tuning. For background on the model's architecture, you can refer to Figure 1 of the ViT paper.

Framework

PyTorch is the required framework to use for this assignment. For resnet, You are asked to define your model layer-by-layer by using the available functions from PyTorch. You are free to use the PyTorch document or other sources to help but do not import pre-defined ResNet definitions. You may find convenient to use hugging face for ViT part.

Dataset

We will use CINIC-10, which contains 270,000 images of resolution 32×32, belonging to 10 classes. The training, validation, and test sets each contain 90,000 images.
You can use this link to download CINIC-10 and `torchvision.datasets.ImageFolder` to load the data, as shown below:

```
from torchvision import datasets, transforms

root = './CINIC-10'

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(
        mean=(0.47889522, 0.47227842, 0.43047404),
        std=(0.24205776, 0.23828046, 0.25874835),
    ),
])

dataset = {
    split: datasets.ImageFolder(
        f'{root}/{split}', transform=transform
    )
    for split in ('train', 'valid', 'test')
}
```

Training

For the ResNet, train the network using the given training data for 20 epochs. For ViT, training for a few (2-3) epochs are enough as it's a pre-trained model. For the main experiment settings, we suggest starting with a mini-batch size of 32, ADAM optimizer with learning rate $\alpha = 0.001$, $\beta_1 = 0.9$ and $\beta_2 = 0.999$. You are free to experiment with other batch size, learning rates and other optimizers. Please use cross entropy loss for your main experiment. Record the error after some interval (for ex. 100 steps of mini-batch) so you can monitor it and plot it to show results. During training, you should test on the validation set at some regular intervals; say every 5 epochs (or 1 epoch for ViT), to check whether the model is overfitting.
Note: To plot the loss function or accuracy, you can use matplotlib, tensorboard or wandb to show the curve.

Test Results

Test the trained network on the test data to obtain classification results and show the results in the form of confusion matrix and classification accuracies for each class. For the confusion matrix you could either write the code on your own, or use scikit-learn to to compute the confusion matrix. (See: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html for more details).

Variations

The above ViT training performs the full model training. Conduct additional experiments where you freeze all the ViT model layer except the last classification head. Compare the results of full-model training with

this. Also compare the ResNet and ViT results.

SUBMISSION

For your submission, include 1) your source code and 2) a report. Please follow the following instructions for preparing your submission.

1. For your main experiment setting, show the evolution of training losses and validation accuracy with multiple steps (training log + curves) for the two networks
2. Show the confusion matrix and per-class classification accuracy for this setting.
3. Show some examples of failed cases, with some analysis if feasible.
4. Compare your results for the two networks and any variations you may have tried.

For the source code, we encourage you to submit the code of the main experiment setting with the variation of the settings mentioned above. For your report, you should include the results of both main experiment settings and those with different experiment variations.

Hints

Following are some general hints on structuring your code, it is not required to follow this template.

1. You need to create
   - *Dataset and DataLoader* The dataset-loading code is provided in the Dataset section. You can use PyTorch's `DataLoader` to organize the datasets into mini-batches. Refer to the PyTorch documentation for details.
   - *Loss Function* calculates the loss, given the outputs of the model and the ground-truth labels of the data.
   - *Model* This is the main model. Data points would pass through the model in the forward pass. In the backward pass, using the backpropagation algorithm, gradients are stored. Please write your own model Class instead using the pre-built one in `torchvision`.
   - *Optimizer* These are several optimization schemes: Standard ones are available in Pytorch; we suggest use of ADAM, though you could also try using the plain SGD. You would be calling these to update the model (after the gradients have been stored).
   - *Evaluation* Compute the predictions of the model and compare with the ground-truth labels of your data. In this homework, we are interested in the top-1 prediction accuracy. A demo code snippet for evaluation is provided below for reference.

```python
import torch

def compute_accuracy(model, dataloader, device='cuda'):
    model.eval()  # Set the model to evaluation mode
    correct_predictions = 0
    total_predictions = 0

    with torch.no_grad():
        for images, labels in dataloader:
            images, labels = images.to(device), labels.to(device)

            outputs = model(images)

            _, predicted_labels = torch.max(outputs, 1)

            correct_predictions += (predicted_labels == labels).sum().item()
            total_predictions += labels.size(0)
```

```
    # Calculate accuracy
    accuracy = correct_predictions / total_predictions * 100
    return accuracy

```

- *Training Loops* This is the main function that will be called after the rest are initialized.

2. The following PyTorch tutorial uses CIFAR-10. Although this assignment uses CINIC-10, you can refer to its examples of defining a loss function, setting up an optimizer, writing the training loop, and evaluating the model.

3. Apart from the above, it is suggested to log your results. You could use TensorBoard (compatible with both PyTorch/TensorFlow) or simply use 'logger' module to keep track of loss values, and metrics.

4. Note on creating batches: It is highly recommended to use the DataLoader class of PyTorch which uses multiprocessing to speed up the mini-batch creation.